



UNIVERSIDAD TECNOLÓGICA DE EL SALVADOR

FACULTAD DE CIENCIAS APLICADAS

Estándares de Programación

DOCENTE: ING. RUBEN ESCOBAR ORTEGON

ACTIVIDAD: ENTREGA 4

TEMA: PLAN DE PRUEBAS

ESTUDIANTES:

NOMBRE	CARNET
CASTRO PORTILLO, CAMILO EDERSON	29-5244-2011
ESTRADA VASQUEZ OSCAR ROLANDO	29-0750-2022
LOPEZ VANEGAS HEBER ALEXANDER	29-1458-2022
REYES RAMIREZ, ALAN ISAI	29-3754-2022
RIVERA RIVAS, MARIO OBED	29-0929-2021

CICLO ACADEMICO: 01 - 2025

FECHA DE ENTREGA: 23de Mayo, 2025

TABLA DE CONTENIDO

TABLA DE CONTENIDO	2
1. PLAN DE PRUEBAS.....	3
1.1 Introducción.....	3
1.2 Objetivos	3
1.2.1 Objetivo General	3
1.2.2 Objetivos Específicos	3
1.3 Alcance	4
1.4 Tipos de Prueba	4
1.5 Ambiente de Pruebas	5
1.6 Herramientas Utilizadas.....	5
1.7 Criterios de Aceptación.....	5
1.7.1 Criterios de Entrada (inicio de pruebas)	5
1.7.2 Criterios de Salida (fin de pruebas)	5
1.8 Estrategia de Pruebas	6
1.9 Conclusión.....	6
2. MATRIZ DE CASOS DE PRUEBA.....	7
3. REGISTRO DE DEFECTOS	19
4. CHECKLIST DE USABILIDAD	23

1. PLAN DE PRUEBAS

1.1 Introducción

El presente documento constituye el Plan de Pruebas correspondiente al proyecto de desarrollo del Sistema de Gestión de Aprendizaje (LMS) para la Escuela de Música "Paganini". Dicho sistema ha sido desarrollado como solución tecnológica para la administración académica de cursos musicales, estudiantes, docentes, evaluaciones y calificaciones, mediante una arquitectura web moderna compuesta por un backend implementado en Spring Boot 3, una base de datos relacional MySQL, y un frontend desarrollado con Angular 20.

El plan de pruebas tiene como finalidad definir, de manera sistemática y estructurada, los procedimientos, criterios y recursos necesarios para verificar que el sistema desarrollado cumple con los requerimientos funcionales y no funcionales establecidos en las etapas de análisis y diseño del proyecto. La correcta ejecución de las pruebas permite asegurar la calidad del software antes de su puesta en producción, reducir el riesgo de errores en producción y garantizar una experiencia adecuada para los distintos usuarios del sistema.

1.2 Objetivos

1.2.1 Objetivo General

Verificar que el Sistema LMS para la Escuela de Música "Paganini" cumple con los requerimientos funcionales y estándares de calidad establecidos, mediante la ejecución sistemática de casos de prueba que validen la correcta operación de todas sus funcionalidades.

1.2.2 Objetivos Específicos

- Validar el correcto funcionamiento del módulo de autenticación y control de acceso al sistema.
- Verificar la integridad de las operaciones CRUD sobre los módulos de cursos, estudiantes, lecciones, evaluaciones, entregas y calificaciones.
- Comprobar la correcta comunicación entre el frontend Angular y el backend Spring Boot a través de la API REST.
- Identificar y documentar defectos, inconsistencias o comportamientos no esperados en el sistema.
- Evaluar la usabilidad e interfaz de usuario del frontend desarrollado con TailwindCSS.
- Verificar la consistencia de los datos almacenados en la base de datos MySQL con las operaciones realizadas desde la interfaz.

1.3 Alcance

El presente plan de pruebas comprende la evaluación de los siguientes módulos y funcionalidades del Sistema LMS:

- Módulo de Autenticación: Login de usuarios (ADMIN, TEACHER, STUDENT, PARENT).
- Módulo de Gestión de Cursos: Creación, edición, visualización y eliminación de cursos.
- Módulo de Gestión de Estudiantes: Registro, edición y consulta de información de alumnos.
- API REST: Verificación de endpoints disponibles en el backend Spring Boot.

Quedan fuera del alcance de este plan: pruebas de carga y estrés, pruebas de seguridad avanzadas (penetration testing), pruebas de compatibilidad en múltiples navegadores y dispositivos móviles, y pruebas de integración con sistemas externos.

1.4 Tipos de Prueba

Tipo de Prueba	Descripción	Herramienta / Técnica
Pruebas Funcionales	Verifican que cada funcionalidad del sistema responde conforme a los requerimientos definidos.	Ejecución manual de casos de prueba
Pruebas de Integración	Validan la correcta comunicación entre el frontend Angular y el backend Spring Boot vía API REST.	Postman — Angular HTTP Client
Pruebas de Usabilidad	Evalúan la experiencia de usuario, la claridad de la interfaz y la navegación del sistema.	Checklist de usabilidad — inspección manual
Pruebas de API REST	Verifican los endpoints del backend: respuestas HTTP, estructura JSON y comportamiento ante errores.	Postman — Swagger UI
Pruebas de Validación de Datos	Comprueban que el sistema valida correctamente campos obligatorios, formatos y restricciones de negocio.	Ejecución manual — casos negativos
Pruebas de Regresión	Aseguran que las correcciones de defectos no generen nuevos errores en funcionalidades previamente validadas.	Re-ejecución de casos de prueba anteriores

1.5 Ambiente de Pruebas

Componente	Detalle
Sistema Operativo	Windows 11 / macOS Ventura
Navegador Web	Google Chrome v124+ / Mozilla Firefox v125+
Backend	Spring Boot 3 — Puerto 8080 — localhost
Frontend	Angular 20 — Puerto 4200 — localhost (ng serve)
Base de Datos	MySQL 8.0 — Base de datos: lms_paganini
IDE / Editor	IntelliJ IDEA 2024 (backend) — Visual Studio Code (frontend)
Datos de Prueba	Base de datos con datos semilla (seed data) cargados mediante scripts SQL de inicialización

1.6 Herramientas Utilizadas

Herramienta	Propósito	Uso en el Proyecto
Postman	Pruebas de API REST	Verificación de endpoints, métodos HTTP y respuestas JSON
Google Chrome DevTools	Depuración frontend	Inspección de red, consola de errores JavaScript y análisis de respuestas HTTP
MySQL Workbench	Gestión de base de datos	Verificación de datos almacenados y validación de integridad referencial
IntelliJ IDEA	Ejecución backend	Revisión de logs del servidor Spring Boot durante la ejecución de pruebas
Captura de pantalla (OS)	Evidencia de pruebas	Documentación visual de resultados obtenidos durante la ejecución de casos de prueba
Microsoft Word / Google Docs	Documentación	Elaboración y presentación del plan de pruebas, matrices y registro de defectos

1.7 Criterios de Aceptación

1.7.1 Criterios de Entrada (inicio de pruebas)

- El backend Spring Boot se encuentra desplegado y accesible en localhost:8080.
- El frontend Angular se encuentra en ejecución mediante ng serve en localhost:4200.
- La base de datos MySQL contiene los datos semilla necesarios para la ejecución de los casos de prueba.
- Los endpoints de la API REST han sido verificados previamente mediante Postman.
- Los casos de prueba han sido revisados y aprobados por el equipo.

1.7.2 Criterios de Salida (fin de pruebas)

- Al menos el 80% de los casos de prueba han sido ejecutados satisfactoriamente.
- No existen defectos de severidad CRÍTICA sin resolver.
- Los defectos de severidad ALTA han sido documentados con plan de corrección.
- Todos los requerimientos funcionales principales tienen al menos un caso de prueba ejecutado.
- El registro de defectos ha sido actualizado con el estado final de cada bug reportado.

1.8 Estrategia de Pruebas

La estrategia de pruebas adoptada para este proyecto combina el enfoque de pruebas de caja negra (black box testing) con inspección manual de la interfaz de usuario. Se ejecutarán primero las pruebas unitarias de los endpoints de la API REST mediante Postman, seguidas de las pruebas funcionales integradas a través del frontend Angular. El proceso se organiza en tres fases:

- Fase 1 — Verificación de API: Se validan todos los endpoints disponibles (/api/users, /api/courses, /api/students, entre otros) mediante solicitudes HTTP directas con Postman, verificando códigos de respuesta (200, 201, 204, 400, 404) y estructura del JSON devuelto.
- Fase 2 — Pruebas Funcionales de Frontend: Se ejecutan los casos de prueba definidos en la Matriz de Pruebas, cubriendo los flujos principales de usuario: login, gestión de cursos, estudiantes, lecciones, evaluaciones, entregas y calificaciones.
- Fase 3 — Revisión de Usabilidad: Se aplica el checklist de usabilidad para evaluar la experiencia general del usuario con el sistema, verificando consistencia visual, mensajes informativos, validaciones y comportamiento responsive.

Los defectos identificados durante la ejecución serán registrados en el Registro de Defectos con severidad, prioridad y estado. Las pruebas de regresión se ejecutarán sobre los módulos afectados una vez que los defectos críticos y de alta prioridad sean corregidos por el equipo de desarrollo.

1.9 Conclusión

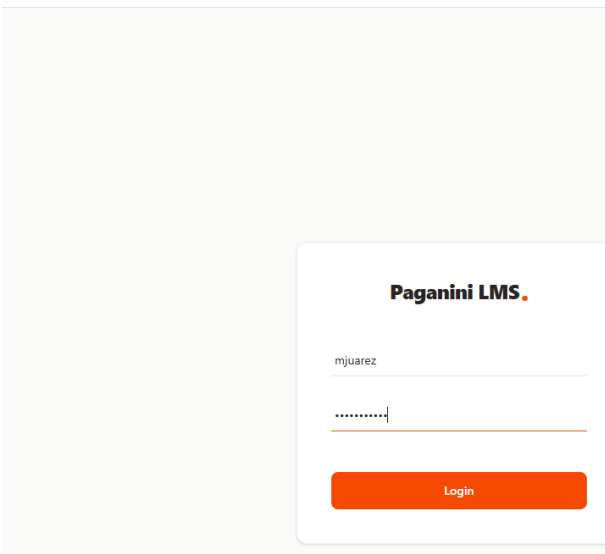
El presente plan de pruebas provee el marco metodológico necesario para asegurar la calidad del Sistema LMS para la Escuela de Música "Paganini". La correcta aplicación de los tipos de prueba definidos, la utilización de las herramientas seleccionadas y el cumplimiento de los criterios de aceptación establecidos permitirán validar de manera objetiva y documentada que el sistema satisface los requerimientos funcionales acordados y ofrece una experiencia de usuario adecuada para los diferentes roles contemplados: ADMIN, TEACHER, STUDENT y PARENT. La generación de evidencia durante la ejecución de los casos de prueba garantizará la trazabilidad completa del proceso de aseguramiento de calidad del proyecto.

2. MATRIZ DE CASOS DE PRUEBA

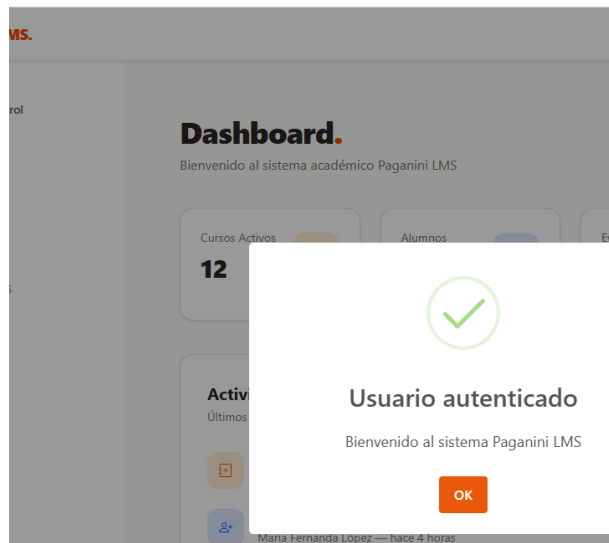
La siguiente matriz presenta los doce (12) casos de prueba definidos para el Sistema LMS de la Escuela de Música "Paganini". Los casos han sido diseñados para cubrir los requerimientos funcionales principales del sistema, incluyendo autenticación, gestión de cursos, estudiantes, lecciones, evaluaciones, entregas y calificaciones.

CASO DE PRUEBA — TC-01: Inicio de sesión con credenciales válidas	
ID del Caso	TC-01
Req. Funcional	RF-01 — Autenticación de usuarios
Nombre del Caso	Inicio de sesión con credenciales válidas
Precondiciones	El sistema se encuentra en ejecución. Existe un usuario registrado con username: mjuarez y password: hashedpass1.
Pasos de Prueba	1. Navegar a http://localhost:4200/login. 2. Ingresar username. 3. Ingresar password. 4. Hacer clic en el botón "Iniciar Sesión".
Resultado Esperado	El sistema redirige al usuario al dashboard principal. El usuario recibe mensaje de éxito de ingreso. El usuario queda registrado en localStorage.
Resultado Obtenido	El sistema redirige correctamente al dashboard. El objeto de usuario queda almacenado en localStorage.
Estado	EJECUTADO
Evidencia	Captura de pantalla del dashboard después del login exitoso. Captura de la consola del navegador mostrando el localStorage con el usuario almacenado.

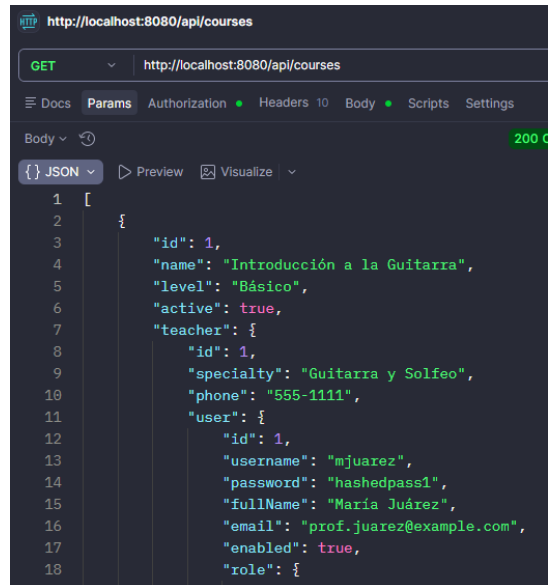
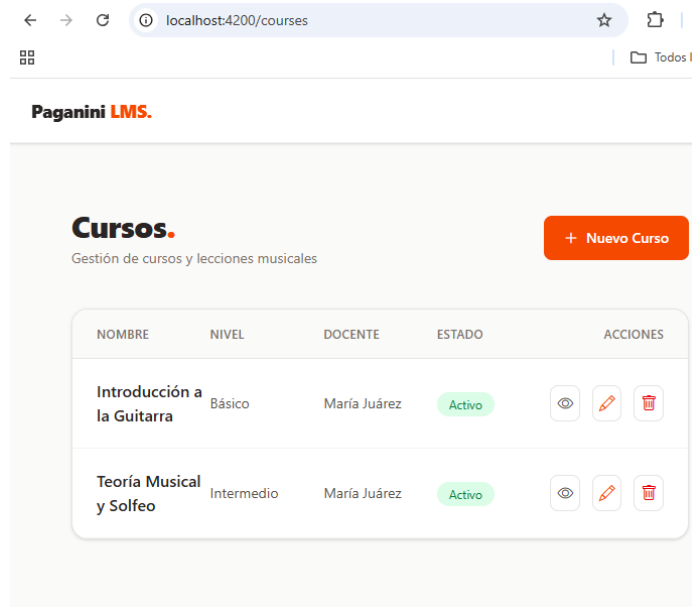
localhost:4200/login



localhost:4200/dashboard



CASO DE PRUEBA — TC-02: Cargar listado de cursos correctamente	
ID del Caso	TC-02
Req. Funcional	RF-02 — Gestión de Cursos
Nombre del Caso	Se carga la lista de cursos correctamente mostrando activos e inactivos.
Precondiciones	Usuario ADMIN autenticado en el sistema. El módulo de cursos está accesible desde el menú lateral.
Pasos de Prueba	1. Navegar a barra lateral. 2. Hacer clic en "Cursos". 3. Verificar carga de lista de cursos creados en base de datos.
Resultado Esperado	La Lista de cursos se carga satisfactoriamente. En el inspector de Chrome se envía mensaje de consola confirmando la carga total.
Resultado Obtenido	Cursos de cargan correctamente. Endpoint api/courses/get funcionando.
Estado	EJECUTADO
Evidencia	Captura del formulario de creación de cursos cargado. Captura de la respuesta 201 en Postman.



CASO DE PRUEBA — TC-03: Crear nuevo curso	
ID del Caso	TC-03
Req. Funcional	RF-02 — Gestión de Cursos
Nombre del Caso	Crear nuevo curso
Precondiciones	Usuario ADMIN autenticado. Existe al menos un curso registrado en el sistema.
Pasos de Prueba	1. Navegar al listado de cursos. 2. Seleccionar botón NUEVO CURSO. 3. Agregar Nombre de Curso 4. Digitar nivel de Curso 5. Seleccionar Profesor de Curso. 6. Recibir notificación de Guardado 7. Actualizar lista automáticamente
Resultado Esperado	Los datos del curso se actualizan en la base de datos. Se muestra mensaje de éxito. El listado refleja los datos modificados. La API recibe un POST a /api/courses con código 200.
Resultado Obtenido	La adición del curso se realizó correctamente. El listado se refrescó con los datos actualizados y se mostró el mensaje de confirmación.
Evidencia	Captura del formulario de edición con datos agregados. Captura del listado actualizado. Captura de la respuesta HTTP 200 en Postman.

Curso.
Crear o editar curso

Nombre del Curso

Solfeo

Nivel

Intermedio

Docente

María Juárez

☒ Curso activo

Cancelar

Guardar



Curso guardado

El curso fue registrado correctamente

OK

Teoría Musical y Solfeo	Intermedio	María Juárez	Activo			
Guitarra Clásica	Intermedia	María Juárez	Activo			
Solfeo	Intermedio	María Juárez	Activo			

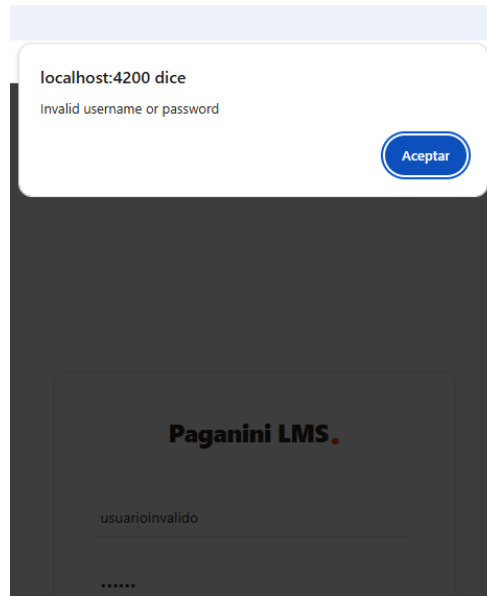
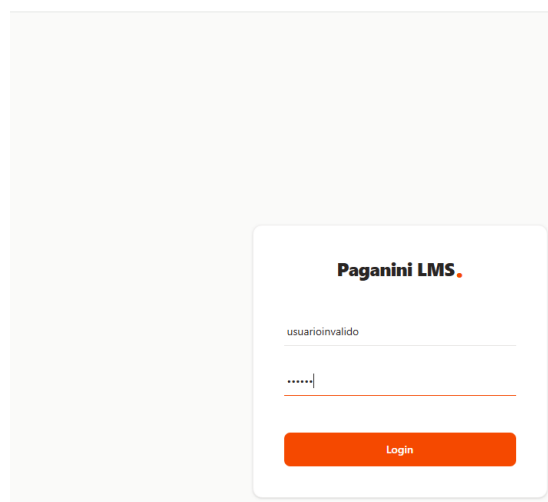
CASO DE PRUEBA — TC-04: Registrar nuevo estudiante en el sistema	
ID del Caso	TC-04
Req. Funcional	RF-03 — Gestión de Estudiantes
Nombre del Caso	Registrar nuevo estudiante en el sistema
Precondiciones	Usuario ADMIN autenticado. Existe al menos un registro de usuario (user) disponible para asignar al estudiante.
Pasos de Prueba	1. Navegar al módulo de Estudiantes. 2. Hacer clic en "Nuevo Estudiante". 3. Completar los campos: fecha de nacimiento, teléfono, estado académico, usuario y padre. 4. Hacer clic en "Guardar".
Resultado Esperado	El estudiante se registra correctamente. La API recibe un POST a /api/students con código 201. El estudiante aparece en el listado de estudiantes.
Resultado Obtenido	El estudiante fue registrado sin errores. El listado de estudiantes se actualizó mostrando el nuevo registro.
Estado	PENDIENTE
Evidencia	Captura de pantalla del formulario de registro completado. Captura del listado de estudiantes con el nuevo registro visible.

CASO DE PRUEBA — TC-05: Crear nueva lección asociada a un curso	
ID del Caso	TC-05
Req. Funcional	RF-04 — Gestión de Lecciones
Nombre del Caso	Crear nueva lección asociada a un curso
Precondiciones	Usuario ADMIN o TEACHER autenticado. Existe al menos un curso activo en el sistema.
Pasos de Prueba	1. Navegar al detalle de un curso. 2. Acceder a la sección de Lecciones. 3. Hacer clic en "Agregar Lección". 4. Ingresar título y fecha de lección. 5. Guardar.
Resultado Esperado	La lección se crea y queda vinculada al curso. La API recibe POST a /api/lessons con código 201. La lección aparece en la lista de lecciones del curso.
Resultado Obtenido	La lección fue creada y desplegada correctamente en el listado de lecciones del curso seleccionado.
Estado	PENDIENTE
Evidencia	Captura del formulario de lección. Captura del listado de lecciones actualizado dentro del detalle del curso.

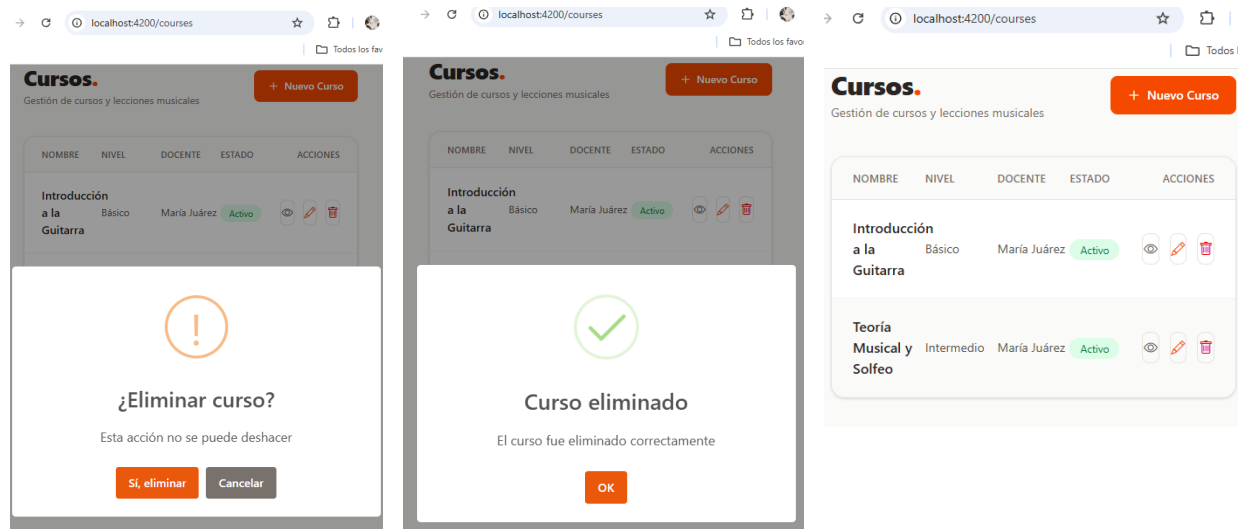
CASO DE PRUEBA — TC-06: Registrar calificación para un estudiante	
ID del Caso	TC-06
Req. Funcional	RF-06 — Registro de Calificaciones
Nombre del Caso	Registrar calificación para un estudiante
Precondiciones	Existe una evaluación registrada. Existe al menos un estudiante con entrega asociada a esa evaluación. Usuario TEACHER autenticado.
Pasos de Prueba	1. Navegar a la evaluación correspondiente. 2. Seleccionar la entrega del estudiante. 3. Ingresar puntaje y comentarios en el formulario de calificación. 4. Hacer clic en "Guardar Calificación".
Resultado Esperado	La calificación se registra correctamente. La API envía POST a /api/grades con código 201. El campo "aprobado" se calcula automáticamente según el puntaje máximo.
Resultado Obtenido	La calificación fue guardada correctamente. El estado "aprobado" fue asignado de forma automática con base en el puntaje registrado.
Estado	PENDIENTE
Evidencia	Captura del formulario de calificación completado. Captura de la respuesta JSON de la API mostrando el campo approved:true.

CASO DE PRUEBA — TC-07: Inicio de sesión con credenciales inválidas	
ID del Caso	TC-07
Req. Funcional	RF-01 — Autenticación de usuarios
Nombre del Caso	Inicio de sesión con credenciales inválidas
Precondiciones	El sistema se encuentra en ejecución. Ningún usuario registrado con username: usuario_falso.
Pasos de Prueba	1. Navegar a http://localhost:4200/login. 2. Ingresar username: usuario_falso. 3. Ingresar password: clave_incorrecta. 4. Hacer clic en "Iniciar Sesión".
Resultado Esperado	El sistema muestra un mensaje de error indicando credenciales incorrectas. No se realiza redirección al dashboard. No se almacena nada en localStorage.
Resultado Obtenido	De acuerdo a lo esperado
Evidencia	Pendiente — Captura de pantalla del mensaje de error de credenciales inválidas.

localhost:4200/login



CASO DE PRUEBA — TC-08: Eliminar un curso del sistema	
ID del Caso	TC-08
Req. Funcional	RF-02 — Gestión de Cursos
Nombre del Caso	Eliminar un curso del sistema
Precondiciones	Usuario ADMIN autenticado. Existe al menos un curso sin enrollments activos.
Pasos de Prueba	1. Navegar al listado de cursos. 2. Localizar el curso a eliminar. 3. Hacer clic en el botón "Eliminar". 4. Confirmar la acción en el diálogo de confirmación.
Resultado Esperado	El sistema solicita confirmación antes de eliminar. Al confirmar, el curso es eliminado. La API envía DELETE a /api/courses/{id} con código 204. El curso desaparece del listado.
Resultado Obtenido	Recurso eliminado de acuerdo a lo previsto
Evidencia	Captura del modal de confirmación. Captura del listado actualizado sin el curso eliminado.



CASO DE PRUEBA — TC-09: Registrar evaluación vinculada a una lección	
ID del Caso	TC-09
Req. Funcional	RF-05 — Gestión de Evaluaciones
Nombre del Caso	Registrar evaluación vinculada a una lección
Precondiciones	Usuario TEACHER autenticado. Existe al menos una lección registrada en el sistema.
Pasos de Prueba	1. Navegar al detalle de una lección. 2. Hacer clic en "Agregar Evaluación". 3. Ingresar título, puntaje máximo y fecha. 4. Guardar la evaluación.
Resultado Esperado	La evaluación queda registrada y vinculada a la lección. La API recibe POST a /api/evaluations con código 201.
Resultado Obtenido	Pendiente de ejecución.
Estado	PENDIENTE
Evidencia	Pendiente — Captura del formulario de evaluación. Captura del listado de evaluaciones de la lección.

CASO DE PRUEBA — TC-10: Registrar entrega con enlace YouTube válido	
ID del Caso	TC-10
Req. Funcional	RF-07 — Entregas de Estudiantes
Nombre del Caso	Registrar entrega con enlace YouTube válido
Precondiciones	Usuario STUDENT autenticado. Existe una evaluación activa a la que el estudiante está inscrito.
Pasos de Prueba	1. Navegar a la evaluación correspondiente. 2. Hacer clic en "Enviar Entrega". 3. Ingresar título, enlace de YouTube válido y comentarios. 4. Guardar.
Resultado Esperado	La entrega queda registrada. La API recibe POST a /api/submissions con código 201. La URL de YouTube se almacena correctamente.
Resultado Obtenido	Pendiente de ejecución.
Estado	PENDIENTE
Evidencia	Pendiente — Captura del formulario de entrega con URL válida. Captura del registro guardado.

CASO DE PRUEBA — TC-11: Consultar calificaciones de un estudiante	
ID del Caso	TC-11
Req. Funcional	RF-08 — Consulta de Calificaciones
Nombre del Caso	Consultar calificaciones de un estudiante
Precondiciones	Usuario STUDENT o PARENT autenticado. Existen calificaciones registradas para el estudiante.
Pasos de Prueba	1. Navegar al módulo de calificaciones o perfil del estudiante. 2. Visualizar las calificaciones registradas por evaluación.
Resultado Esperado	El sistema lista todas las calificaciones del estudiante con puntaje, estado de aprobación y comentarios del docente.
Resultado Obtenido	Pendiente de ejecución.
Estado	PENDIENTE
Evidencia	Pendiente — Captura del listado de calificaciones del estudiante.

CASO DE PRUEBA — TC-12: Validación de campos obligatorios al crear curso	
ID del Caso	TC-12
Req. Funcional	RF-09 — Validación de Formularios
Nombre del Caso	Validación de campos obligatorios al crear curso
Precondiciones	Usuario ADMIN autenticado. Se accede al formulario de creación de curso.
Pasos de Prueba	1. Navegar al formulario de creación de curso. 2. Dejar en blanco el campo "Nombre del Curso". 3. Se muestran mensajes de alerta de campos obligatorios 4. Se inhabilita el botón "Guardar".
Resultado Esperado	El sistema muestra mensajes de validación indicando que el nombre del curso es obligatorio. No se realiza ninguna llamada a la API.
Resultado Obtenido	Pendiente de ejecución.
Evidencia	Pendiente — Captura del formulario con mensajes de validación visibles.

Curso.

Crear o editar curso

Nombre del Curso

Ejemplo: Piano Básico

Campo requerido

Nivel

Beginner / Intermediate

Campo requerido

Docente

Selecciona un docente

Campo requerido

Docente

Selecciona un docente

Campo requerido

☒ Curso activo

Cancelar

 Guardar

Botón Guardar inhabilitado hasta que se completen los campos requeridos.

3. REGISTRO DE DEFECTOS

El siguiente registro documenta los defectos identificados durante la ejecución de las pruebas funcionales y de usabilidad del Sistema LMS. Cada defecto incluye descripción, nivel de severidad, prioridad de atención, módulo afectado y estado actual.

<i>ID BUG</i>	<i>Descripción del Defecto</i>	<i>Severidad</i>	<i>Prioridad</i>	<i>Módulo Afectado</i>	<i>Estado</i>
BUG-01	Al intentar crear un nuevo curso, la aplicación lanza el error: "Row was already updated or deleted by another transaction for entity". El sistema no permite guardar el curso y el registro no se crea en la base de datos.	Alta	Alta	Gestión de Cursos	ABIERTO
BUG-02	En el módulo de Alumnos, el botón "Nuevo Alumno" no responde al hacer clic. Adicionalmente, en el listado de alumnos los botones de acción Detalle, Editar y Eliminar no ejecutan ninguna función.	Alta	Alta	Gestión de Estudiantes	ABIERTO
BUG-03	El componente de Detalle del Estudiante no renderiza la información del alumno seleccionado. En su lugar se muestra únicamente un mensaje genérico, sin datos del estudiante.	Alta	Alta	Gestión de Estudiantes	ABIERTO
BUG-04	Al iniciar sesión, el header del sistema muestra las iniciales estáticas "CC" en lugar de las iniciales reales del usuario autenticado. El sistema no extrae las iniciales del objeto de usuario almacenado en localStorage.	Media	Media	Autenticación / Header	ABIERTO
BUG-05	El sistema no valida la existencia previa de un curso antes de crearlo. Esto permite registrar dos cursos con exactamente el mismo nombre, nivel y docente, generando duplicados en la base de datos y en el listado de cursos.	Media	Alta	Gestión de Cursos	ABIERTO

BUG 1: NO PERMITE CREAR CURSO NUEVO. APLICACIÓN LANZA MENSAJE DE ERROR. *Row was already updated or deleted by another transaction for entity*

Curso.
Crear o editar curso

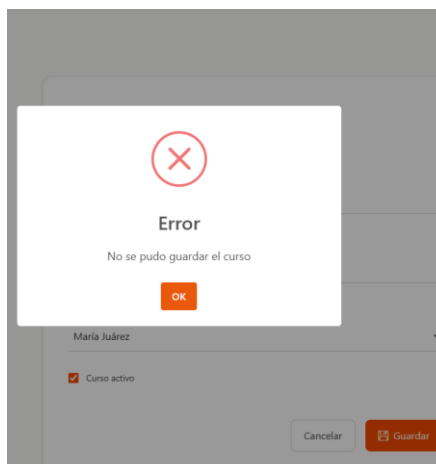
Nombre del Curso
Guitarra

Nivel
Intermedia

Docente
María Juárez

☒ Curso activo

Cancelar Guardar



BUG 2: En la pantalla de Alumnos, al intentar crear un nuevo alumno el botón no responde. En el listado de Alumnos las acciones de Detalle, Editar y Eliminar no están funcionando.

localhost:4200/students

MS.

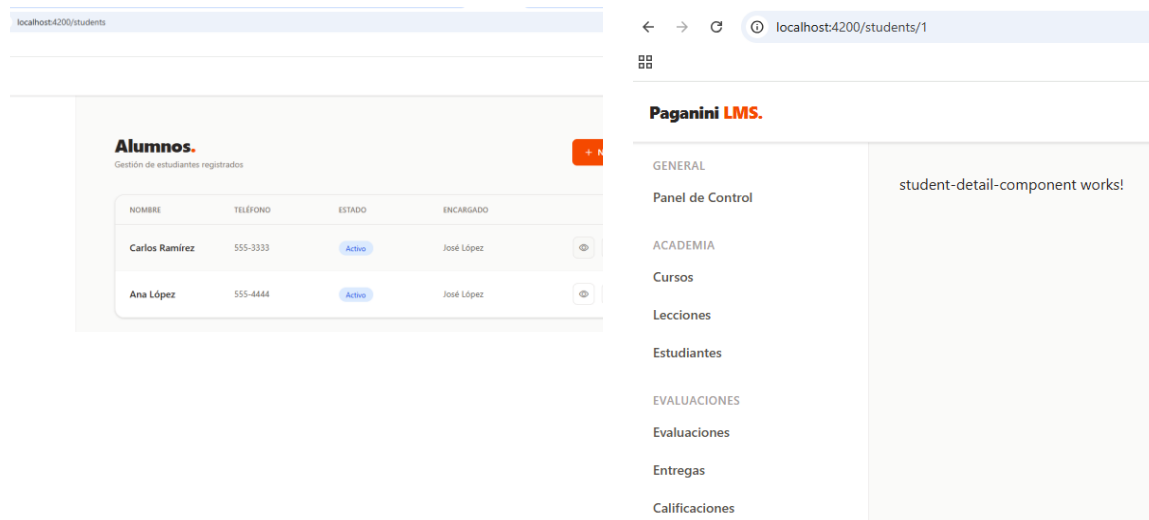
rol

Alumnos.
Gestión de estudiantes registrados

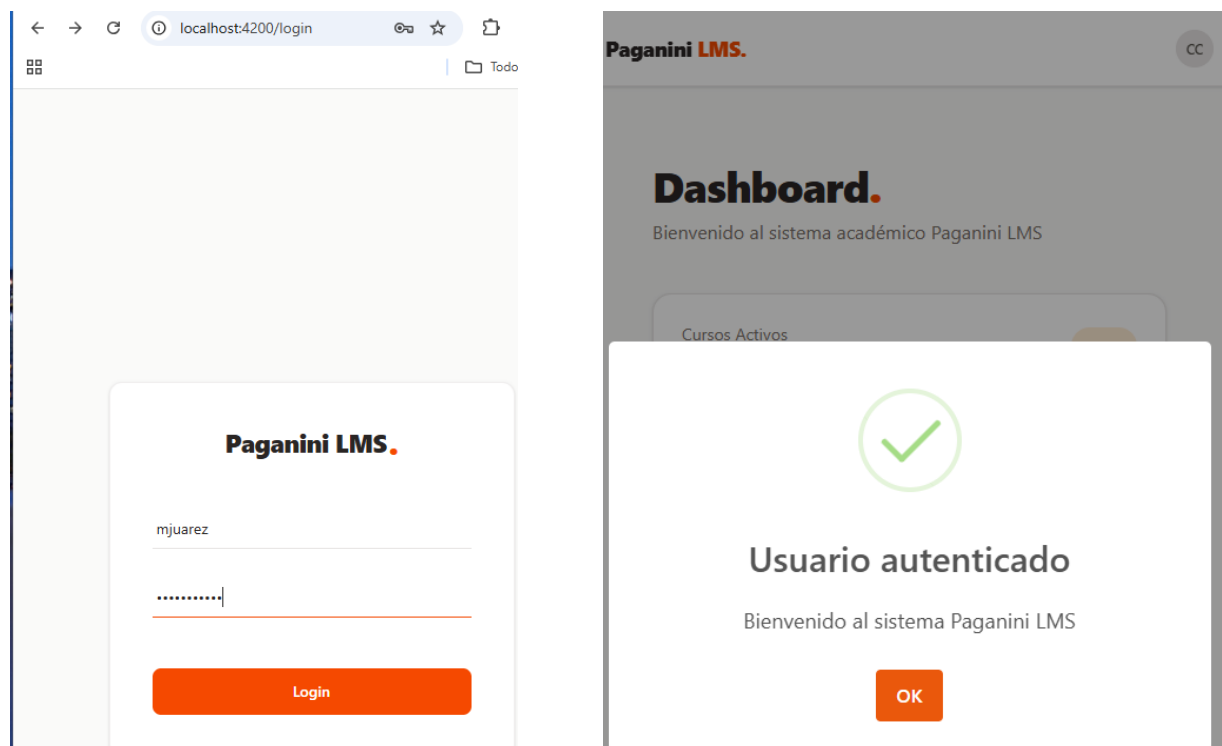
+ Nuevo Alumno

NOMBRE	TELÉFONO	ESTADO	ENCARGADO	ACCIONES
Carlos Ramírez	555-3333	Activo	José López	
Ana López	555-4444	Activo	José López	

BUG 3: El detalle del Estudiante no está mostrando el componente. No se muestran detalles del estudiante si no mensaje genérico.



BUG 4: Al iniciar sesión, no se muestran las iniciales correctas del usuario autenticado. Se muestran iniciales estáticas CC en header.



BUG 5:

No se valida la existencia de un curso antes de crearlo. Esto permite que dos cursos exactamente iguales se guarden en la base de datos. Las capturas muestran que se crean dos cursos iguales y ambos se guardan a la base de datos y se muestran en la lista.

Curso.
Crear o editar curso

Nombre del Curso

Teoría Musical y Solfeo

Nivel

Intermedio

Docente

María Juárez

☒ Curso activo

Cancelar

Guardar



Curso guardado

El curso fue registrado correctamente

OK

Cursos.
Gestión de cursos y lecciones musicales

+ Nuevo Curso

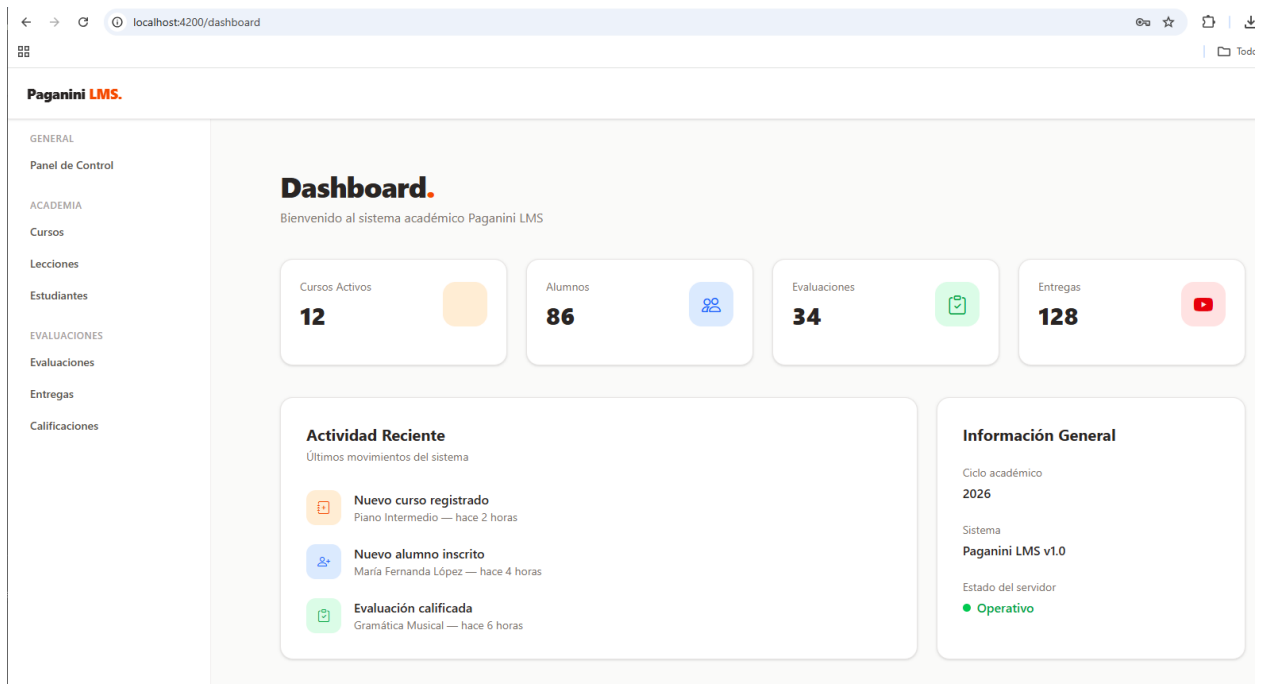
NOMBRE	NIVEL	DOCENTE	ESTADO	ACCIONES
Introducción a la Guitarra	Básico	María Juárez	Activo	  
Teoría Musical y Solfeo	Intermedio	María Juárez	Activo	  
Teoría Musical y Solfeo	Intermedio	María Juárez	Activo	  

4. CHECKLIST DE USABILIDAD

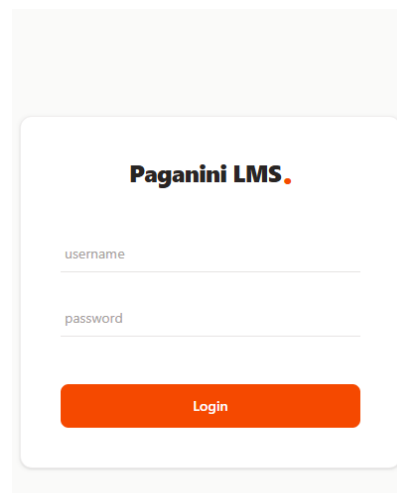
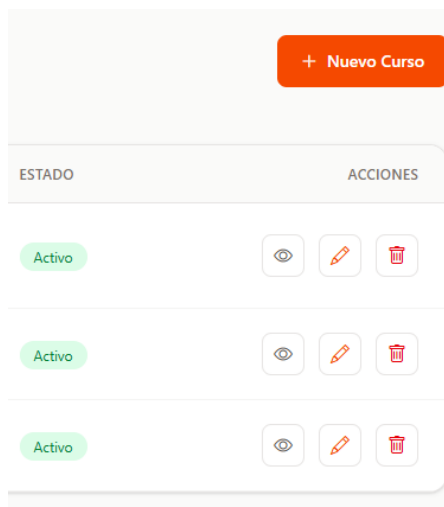
La siguiente lista de verificación evalúa la usabilidad del Sistema LMS para la Escuela de Música "Paganini", considerando criterios estándar de experiencia de usuario, consistencia visual, accesibilidad básica y comportamiento general de la interfaz.

#	Ítem de Usabilidad	Estado	Observaciones
1	Navegación intuitiva	Cumple	El sidebar lateral provee acceso rápido a todos los módulos del sistema con iconos descriptivos. La jerarquía de navegación es clara.
2	Validación de campos obligatorios	No cumple	BUG-01: Varios formularios del sistema (ej. creación de cursos) no muestran mensajes de validación cuando los campos obligatorios se dejan vacíos.
3	Consistencia visual de la interfaz	Cumple	El diseño mantiene coherencia visual en toda la aplicación utilizando la paleta stone + orange de TailwindCSS. Los componentes son homogéneos en todos los módulos.
4	Mensajes de retroalimentación claros	No cumple	BUG-06: Los mensajes de error del backend no se despliegan al usuario. Los mensajes de éxito existen en algunos módulos pero no son consistentes en todos.
5	Acciones visibles y reconocibles	Cumple	Los botones de acción (Nuevo, Editar, Eliminar, Guardar, Cancelar) son claramente visibles y utilizan colores diferenciados (verde para guardar, rojo para eliminar).
6	Confirmación antes de eliminar registros	No cumple	BUG-02: El sistema no solicita confirmación antes de ejecutar la eliminación de registros, lo que representa un riesgo de pérdida accidental de datos.
7	Diseño responsive básico	No cumple	BUG-05: En resoluciones móviles (< 768px) se presentan problemas de superposición en formularios. El layout está optimizado principalmente para escritorio.
8	Colores con contraste legible	Cumple	La combinación de colores stone y orange sobre fondo claro garantiza un contraste adecuado para la lectura del texto. Los encabezados son claramente distinguibles.
9	Tiempo de respuesta adecuado	Cumple	Las operaciones CRUD responden en menos de 1 segundo en entorno local. No se observaron bloqueos o tiempos de espera excesivos durante las pruebas.
10	Indicadores de carga (loading)	No cumple	El sistema no muestra indicadores de carga durante las solicitudes a la API REST. El usuario no recibe retroalimentación visual mientras la operación se procesa.

11	Actualización automática de listados	No cumple	BUG-04: Tras crear o editar un registro, el listado no se refresca automáticamente. Se requiere recarga manual de la página para ver los cambios.
12	Accesibilidad básica del formulario	Cumple	Los campos de formulario incluyen etiquetas (labels) asociadas correctamente. El orden de tabulación entre campos es coherente y facilita la entrada de datos con teclado.



Interfaz de usuario. Breakpoint LG: desde 992px hasta 1199



Diseño UX/UI coherente a nivel visual